

Propozycje tematów pracy magisterskiej

Prace magisterskie (wniosek tytułu do promotora) — dwa niezależne tematy na bazie projektu inżynierskiego SmartRecommender

Rozszerzenie pracy inżynierskiej *SmartRecommender*
o warstwę badawczą, eksperymenty i — opcjonalnie — panel diagnostyczny w Django Admin

Autorzy: Dawid Olko, Piotr Smoła
Promotor: dr hab. Urszula Bentkowska
Jednostka: Uniwersytet Rzeszowski, WNST
Repozytorium: github.com/dawidolko/SmartRecommender
Data: 2026-05-02

Cel dokumentu: zestaw **propozycji tematów pod prace magisterskie** (dwie osobne prace) oraz opis planowanej infrastruktury badawczej — bez konieczności pisania aplikacji od zera.

Opracowanie po analizie repozytorium, założeń promotora i wcześniejszych dokumentów (`propozycje.md`, `propozycje_tematow_magisterskich_v2.pdf`).

Spis treści

Streszczenie	3
1 Punkt wyjścia — stan obecny po pracy inżynierskiej	3
2 Co należy poprawić w istniejącym projekcie zanim zaczniemy badania	4
2.1 Plan naprawczy — nowa komenda <code>seed_research</code>	5
2.1.1 Co to znaczy „persona”? — wyjaśnienie prostym językiem	5
2.1.2 Cel badawczy nowego seedera — ok. 100–200 klientów i pełne ścieżki zakupowe	6
2.1.3 Persony użytkowników — przykładowe typy w kodzie <code>seed_research</code>	6
2.1.4 Skala danych — kompromis jakość vs czas	8
2.1.5 Spójność danych (kluczowe ulepszenia)	8
2.1.6 Migracje schematu bazy	8
2.1.7 Wspólny moduł oceny	9
2.2 Duże modele językowe (LLM) i RAG — jak to działa w <i>SmartRecommender</i>	9
2.2.1 Typowe dwa tryby użycia LLM w rekomendacji	10
2.2.2 Co konkretnie porównujemy (przykładowe zestawy eksperymentów)	10
2.2.3 Biblioteki i usługi (Python) — orientacyjny stos	10
2.2.4 Czy da się “za darmo”?	11
2.2.5 Przepływ żądania (uproszczony)	11
2.3 Panel badań w Django Admin — wyniki, uruchamianie eksperymentów, dobór metody .	11
2.3.1 Co rejestrujcie w adminie	11
2.3.2 GUI zamiast “tylko <code>manage.py</code> ”	11
2.3.3 Zmiana metody w runtime — “auto” wg najlepszego wyniku	12
2.3.4 Nowa “zakładka” w adminie	12
3 Katalog propozycji tematów magisterskich	13
3.1 Szybki przegląd — pełna lista propozycji	13
3.2 Dawid Olko — propozycje szczegółowe	14
3.3 Twój kod z inżynierki — gdzie co masz (mapa pod obronę)	14
3.3.1 Łączenie metod rekomendacji versus metody pojedyncze (D1)	14
3.3.2 Wpływ hiperparametrów na jakość i czas — CF, Apriori, sentyment (D2)	15
3.3.3 Duży model językowy jako główny kanał rekomendacji (D3; porównanie z inżynierką opcjonalne)	15
3.3.4 Jakość rekomendacji przy profilu z danych jawnych (opinie, oceny) (D4)	17
3.3.5 Skalowalność collaborative filtering i reguł asocjacyjnych (D5)	17
3.3.6 Kontrolowany, powtarzalny zbiór danych pod ewaluację metod (D6)	18
3.3.7 Cold start użytkownika lub produktu (D7)	19
3.3.8 Jednolity pipeline ewaluacji offline jako element pracy (D8)	19
3.3.9 LLM i RAG wyłącznie nad treścią katalogu (D9)	20
3.3.10 Wyjaśnialność wybranego rankera (D10)	21
3.3.11 Bezpieczeństwo i walidacja wyjść LLM w Django (D11)	22
3.3.12 Uczciwość rankingu w podgrupach person (D12)	23

3.4	Piotr Smoła — propozycje szczegółowe	23
3.4.1	Sieć neuronowa łącząca wiele metod z kontekstem użytkownika (P1)	23
3.4.2	Segmentacja użytkowników (K-Means, DBSCAN) i strategia per segment (P2) . . .	23
3.4.3	Hyperparameter tuning — CBF, silnik rozmyty, modele probabilistyczne (P3) . . .	24
3.4.4	Warstwa integracji LLM (API, walidacja, koszt, niezawodność) (P4)	24
3.4.5	Rekomendacje z danych ukrytych (interakcje, ewentualnie dwell) (P5)	25
3.4.6	Skalowalność CBF, logiki rozmytej i ścieżek probabilistycznych (P6)	25
3.4.7	Wnioskowanie rozmyte typu drugiego (IT2-FLS) wobec Mamdani T1 z inżynierki (P7)	26
3.4.8	Meta-uczenie prostsze od głębokiej sieci (np. gradient boosting) (P8)	26
3.4.9	Ranker na embeddingach tekstu produktów (P9)	26
3.4.10	Graf współwystąpień produktów z koszyków (P10)	27
3.4.11	Reprodukowalność eksperymentów i automatyzacja (P11)	27
3.4.12	Dryf jakości i monitoring rankingu (P12)	27
4	Przykładowa realizacja wybranych tematów (głęboki opis)	29
4.1	Praca Dawida Olko — CF + Sentyment + Apriori	29
4.2	Praca Piotra Smoły — CBF + Fuzzy + Probabilistic	33
5	Wspólny rdzeń projektu	37
5.1	seed_research — szczegółowy plan	37
5.2	home/experiments/ — wspólny moduł	37
5.3	Tracking dwell time (frontend + backend) — opcjonalny	38
5.4	Panel diagnostyczny eksperymentów (opcjonalnie)	38
6	Tabelaryczne porównanie tematów	39
7	Harmonogram realizacji (12 miesięcy)	39
8	Ryzyka i plany awaryjne	40
9	Założenia środowiskowe — część LLM (Olko), ustalone przez autora	40
10	Pytania do uzgodnienia z promotorem	40

Streszczenie

Niniejszy dokument dotyczy dwóch równoległych prac magisterskich jako **rozbudowy istniejącego SmartRecommender**: bez przepisywania frontu i backendu od zera, z dodaniem warstwy badawczej (dane, eksperymenty, nowe moduły tam, gdzie to potrzebne). Kod i interfejs z pracy inżynierskiej pozostają punktem wyjścia, ale **nie każdy temat musi** kończyć się porównaniem z któryś z metod z inżynierki — część propozycji to np. samodzielny tor LLM/RAG, nowy ranker, jakość usługi, graf towarów, monitoring; porównanie z istniejącym kodem jest *możliwe*, lecz *nieobowiązkowe*.

W dokumencie zawarto m.in. **stos technologiczny dla LLM/RAG** (sekcja 2.2) oraz koncepcję **panelu badań w Django Admin** (sekcja 2.3) — sens: wyniki eksperymentów i wybór strategii rekomendacji widoczne w GUI, a nie wyłącznie w terminalu.

W dokumencie zawarto **katalog wielu propozycji** dla każdego autora: najpierw **tabele skrótowe** (sekcja 3.1) — żeby od razu widać było pełną listę identyfikatorów D1–D12 i P1–P12 — potem rozwinięcia z celem, pytaniem badawczym, danymi i metrykami. **Dla Dawida** dodano mapę istniejącego kodu z inżynierki (sekcja 3.3) i przy każdym temacie D1–D12 blok „jak to powiedzieć na obronie” + „co w repozytorium”. Sekcja 4 to wyłącznie *jeden* przykład głębokiego opracowania (hybrydy + LLM; segmentacja + IT2); można go zastąpić inną pozycją z katalogu.

Olko (przykład w sekcji 4): hybrydy CF + sentyment + Apriori oraz LLM.

Smola (przykład): segmentacja użytkowników oraz IT2-FLS wobec Mamdani T1.

Propozycje można zestawiać z typowymi punktami u promotora (kombinacje, tuning, LLM, sieć łącząca, segmentacja, jawne vs ukryte, skalowalność, IT2); w katalogu są też tematy **poza** tym zestawem (np. RAG, graf, fairness, monitoring).

1. Punkt wyjścia — stan obecny po pracy inżynierskiej

W ramach pracy inżynierskiej zrealizowanej wspólnie przez autorów powstał działający system rekomendacji produktów *SmartRecommender* w architekturze:

- **Backend**: Django 5.1 + Django REST Framework + PostgreSQL 16
- **Frontend**: React 18 (SPA) z uwierzytelnianiem JWT
- **Konteneryzacja**: Docker Compose (3 usługi: db, backend, frontend)
- **Baza**: 24 tabele, ok. 70 endpointów REST, panel admina i klienta

Zaimplementowano sześć metod rekomendacyjnych (oraz kilka podmodułów, które dla potrzeb badań traktujemy jako 9 niezależnych kanałów rankingowych):

Lp.	Metoda / kanał	Autor	Lokalizacja w kodzie
1	Collaborative Filtering (Item-Based, Adjusted Cosine)	Olko	home/recommendation_views.py
2	Content-Based Filtering (Cosine + TF-IDF)	Smoła	home/recommendation_views.py
3	Reguły asocjacyjne — Apriori z bitmap pruning	Olko	home/custom_recommendation_engine.py
4	Logika rozmyta Mamdani typu pierwszego	Smoła	home/fuzzy_logic_engine.py
5	Wyszukiwanie rozmyte (typo tolerance)	Smoła (poboczne)	home/sentiment_views.py
6	Analiza sentymentu (słownikowa, Liu 2012)	Olko	home/custom_recommendation_engine.py
7	Łańcuchy Markowa pierwszego rzędu	Smoła	home/probabilistic_views.py
8	Naiwny klasyfikator Bayesa	Smoła	home/probabilistic_views.py
9	Analiza sezonowości (komponent temporalny)	Smoła	home/seasonal_views.py

Tabela 1: Metody rekomendacyjne zaimplementowane w pracy inżynierskiej.

Każda metoda ma osobny endpoint debugujący (np. `/api/collaborative-filtering-debug/`), co ułatwia walidację w pracy magisterskiej.

2. Co należy poprawić w istniejącym projekcie zanim zaczniemy badania

Istniejący seeder (`backend/home/management/commands/seed.py`, ~1.4 MB, ~17 300 linii) generuje dane w sposób, który nie wystarcza do rzetelnej oceny metod rekomendacyjnych. Wszystkie poniższe problemy zostały zweryfikowane bezpośrednio w kodzie:

Problem	Aktualny stan	Konsekwencja dla badań
Mało użytkowników	5 admin + 15 client (seed_users(), l. 297)	Za mało, by liczyć wiarygodne metryki Precision@K
Losowe interakcje	UserInteraction seedowany dorywczo	Brak realistycznego sygnału ukrytego
Losowe opinie	60 szablonów bez związku z produktem (l. 16606)	Sentyment per-produkt niewiarygodny
Losowe daty	random.randint(1,365) dni wstecz (l. 16803)	Brak temporalnego split
Losowa probablistyka	random.uniform() w analytics.py	Modele probabilistyczne to atrapa, nie ML
Brak typów klientów (persony)	Klienci kupują wszystko losowo	Brak struktury, której metody mogłyby się „uczyć”
Brak czasu na karcie	UserInteraction ma timestamp, brak duration_ms	Nieemożliwe „jawne vs ukryte” w sensie dwell
Brak metadanych eksp.	Brak tabeli wyników	Trzeba szukać metryk po logach konsoli

Tabela 2: Zidentyfikowane problemy obecnego seedera, zweryfikowane w kodzie.

2.1. Plan naprawczy — nowa komenda seed_research

Rozwiązanie: **nowa komenda** `python manage.py seed_research` w pliku `backend/home/management/commands/seed_research.py`

Istniejący `seed.py` pozostaje **bez zmian** jako kompatybilność wsteczna z aplikacją produkcyjną i panelem demo.

2.1.1. Co to znaczy „persona”? — wyjaśnienie prostym językiem

Nie chodzi o żadną osobną funkcję w sklepie dla prawdziwego klienta i nie musisz tego jakoś „widzieć” w Reactcie.

Persona = etykieta typu kupującego przy sztucznych kontach testowych. Wyobraź sobie, że zamiast losować 150 klientów, którzy kupują przypadkowe produkty z całego katalogu, mówisz generatorowi danych:

„Tych 40 użytkowników nazwij gamer — niech w 80% kupują z kategorii gaming i peryferiów.
Tych 35 — office_worker — niech kupują biuro i drukarki.
Itd.”

W bazie i tak jest zwykły User jak każdy inny; dodatkowy wiersz (np. model `UserPersona`) zapisuje tylko **string** typu `gamer` / `office_worker`, żebyś **wiedział**, dlaczego ten użytkownik ma takie zamówienia i żebyś mógł liczyć metryki osobno dla takich grup (fairness, segmentacja).

Po co to komu: metody jak CF i Apriori potrzebują **powtarzalnych schematów** (ludzie podobni do siebie kupują podobne rzeczy). Jeśli koszyki są całkiem losowe, algorytm nie ma sensu i obrona wygląda źle. Persony to sposób, żeby **ustrukturyzować** losowość — nadal syntetyczne dane, ale przypominające prawdziwy świat (ktoś hobby, ktoś biuro), a nie szum telewizyjny.

Synonim na obronie: zamiast „persony” możesz powiedzieć *typy zachowań klientów w wygenerowanych danych* lub *profile testowe* — to jest to samo.

2.1.2. Cel badawczy nowego seedera — ok. 100–200 klientów i pełne ścieżki zakupowe

Chodzi o to, by **dokładając** do projektu osobny strumień danych (typowo: po jednorazowym „bazowym” seedzie lub w środowisku tylko-lab) uzyskać **znacznie większą, sensowną próbę** niż 15 klientów demo:

- **Skala klientów badawczych:** ok. 100–200 kont `client` (parametr CLI, np. `-clients 150`), każdy przypisany do **persony** zachowaniowej (`UserPersona`) — wyjaśnienie słowa „persona”: sekcja 2.1.1; dzięki temu macie różne wzorce koszyków, a nie jeden losowy rozkład.
- **Nowe zamówienia:** dla każdego takiego użytkownika wielokrotne **zamówienia w czasie** (np. średnio 5–25 na osobę, zależnie od osoby), z pozycjami na **różnych produktach** z katalogu — głównie z kategorii zgodnych z personą, część jako eksploracja (inna kategoria), żeby były realistyczne współwystąpienia i długi ogon.
- **Opinie powiązane z zakupem:** nowe rekordy `Opinion` (tekst/ocena) dla produktów faktycznie kupionych przez danego użytkownika; czas powstania opinii **po** dacie zamówienia; sentyment spójny z tym, czy zakup pasuje do osoby (żeby analiza sentymentu i tekst miały sens metodyczny).
- **Interakcje jawne/ukryte:** dla części ścieżek — `UserInteraction` (`view`, `click`, itd.) **przed** zakupem, z sensowną kolejnością czasową; opcjonalnie `duration_ms` przy oglądaniu karty produktu (do badań dwell / implicit), jeśli włączycie migrację i logowanie z frontu.
- **Powtarzalność:** stały `-random-seed` + zapis wersji seeda w manifestcie / polu `seed_research_version` przy eksperymencie — identyczna baza po ponownym uruchomieniu komendy.

Efekt liczbowy w docelowej konfiguracji (patrz tabela poniżej): rząd **tysięcy** opinii, **tysięcy** zamówień, **dziesiątek tysięcy** interakcji — to już podstawa pod stabilne $\text{Precision}@K$, $\text{NDCG}@K$, podział czasowy train/test, segmentację i tuning, bez budowania sklepu od zera.

2.1.3. Persony użytkowników — przykładowe typy w kodzie `seed_research`

Poniżej **konkretna lista** (5 nazw + parametry), którą możesz mieć w słowniku Pythona. To są tylko **przykłady** etykiet z sekcji 2.1.1 — liczba i nazwy mogą być inne po uzgodnieniu w zespole.

```
PERSONAS = {
    "gamer": {
        "preferred_categories": ["computers.gaming", "components.graphics",
                                "peripherals.mice", "peripherals.headphones",
                                "peripherals.keyboards", "gaming.consoles"],
        "avg_basket_size": 2.5,
        "price_sensitivity": "medium",
        "review_propensity": 0.6,          # P(opinia po zakupie)
        "active_hour": (18, 24),
        "dwell_time_mean_ms": 45000,
    },
    "office_worker": {
        "preferred_categories": ["computers.office", "laptops.office",
                                "peripherals.printers", "furniture.desks", "office.accessories"],
        "avg_basket_size": 1.5,
        "price_sensitivity": "high",
        "review_propensity": 0.2,
```

```

    "active_hour": (6, 12),
    "dwell_time_mean_ms": 25000,
  },
  "content_creator": {
    "preferred_categories": ["peripherals.microphones", "peripherals.webcams",
    "components.graphics", "peripherals.monitors",
    "cameras.stabilizers", "peripherals.headphones"],
    "avg_basket_size": 3.0,
    "price_sensitivity": "low",
    "review_propensity": 0.8,
    "active_hour": (12, 18),
    "dwell_time_mean_ms": 60000,
  },
  "gift_buyer": {
    "preferred_categories": ["wearables.watches", "electronics.tablets",
    "electronics.phones", "drones", "gadgets"],
    "avg_basket_size": 1.2,
    "review_propensity": 0.4,
    "seasonality": {"11": 2.5, "12": 3.0, "2": 1.8, "5": 1.5},
  },
  "tech_enthusiast": {
    "preferred_categories": ["components.processors", "components.motherboards",
    "components.memoryRam", "components.disks", "components.cooling"],
    "avg_basket_size": 4.0,
    "price_sensitivity": "low",
    "review_propensity": 0.9,
    "active_hour": (0, 6),
    "dwell_time_mean_ms": 75000,
  },
}

```


2.1.4. Skala danych — kompromis jakość vs czas

Parametr	Aktualnie	seed_research	CLI argument
Klienci (badawczy)	15	100–200 (domyślnie 150)	-clients 150
Admini	5	5	-admins 5
Produkty	~500	~500	-products 500
Zamówienia	10/klient	5–25/klient (persona)	-orders-per-client-mean 12
Interakcje	brak	30–150/klient	-interactions-per-client-mean 80
Opinie	2–5/produkt	0–8/produkt	(auto)
Okno czasowe	365 dni	730 dni	-days-back 730
Seed losowości	brak	42 (powtarzalność)	-random-seed 42

Tabela 3: Skala danych w nowym seederze badawczym (kluczowy skok: **100–200** aktywnych klientów z personami, wieloma zamówieniami i opiniami).

Liczby orientacyjne (dla 150 klientów, średnio 80 interakcji na osobę, średnio 12 zamówień): ok. 12 000 interakcji + rząd 2 000 zamówień + kilka tysięcy opinii — wystarcza na wiarygodne Precision@10 i NDCG@10 przy sensownym teście (reguła praktyczna: dużo więcej niż przy 15 kontach demo).

2.1.5. Spójność danych (kluczowe ulepszenia)

1. **Zamówienia z kategorii zgodnych z personą:** 80% produktów z preferowanych, 20% eksploracji.
2. **Sentyment opinii skorelowany z dopasowaniem do osoby:** gamer kupujący kartę graficzną ma $\text{sentiment_score} \approx +0.7$; office_worker kupujący tę samą kartę przez przypadek ≈ -0.2 .
3. **Opinie po zakupie, nie przed:** `Opinion.created_at > Order.date_order`.
4. **Interakcje przed zakupem:** 3–15 wpisów `UserInteraction` (`view` → `click` → `add_to_cart` → `purchase`) z timestampami przed zamówieniem.
5. **Sezonowość:** `gift_buyer` 3× częściej w grudniu, `tech_enthusiast` boom w listopadzie (Black Friday) i sierpniu.

2.1.6. Migracje schematu bazy

Dodatki do `home/models.py`:

```
class UserInteraction(models.Model):
    # ... istniejące pola: user, product, interaction_type, timestamp ...
    duration_ms = models.PositiveIntegerField(null=True, blank=True,
        help_text="Czas spędzony na karcie (tylko dla 'view')")

class Opinion(models.Model):
    # ... istniejące pola ...
    created_at = models.DateTimeField(auto_now_add=True, null=True, blank=True)
```

```

class UserPersona(models.Model):
    """Powiązanie testowych użytkowników z personą    tylko dane badawcze."""
    user = models.OneToOneField(User, on_delete=models.CASCADE)
    persona_name = models.CharField(max_length=50)
    class Meta:
        db_table = 'research_user_persona'

class ExperimentRun(models.Model):
    """Tabela logująca wyniki eksperymentów."""
    experiment_id = models.CharField(max_length=100, unique=True)
    method_or_combination = models.CharField(max_length=200)
    parameters_json = models.JSONField()
    seed_research_version = models.CharField(max_length=20)
    precision_at_10 = models.FloatField()
    recall_at_10 = models.FloatField()
    ndcg_at_10 = models.FloatField()
    map_score = models.FloatField()
    coverage = models.FloatField()
    diversity = models.FloatField()
    train_size = models.PositiveIntegerField()
    test_size = models.PositiveIntegerField()
    duration_seconds = models.FloatField()
    created_at = models.DateTimeField(auto_now_add=True)
    notes = models.TextField(blank=True)
    class Meta:
        db_table = 'research_experiment_run'
        ordering = ['-created_at']

```

python manage.py makemigrations && python manage.py migrate — bez breaking changes do produkcji, tylko nowe pola opcjonalne.

2.1.7. Wspólny moduł oceny

Plik backend/home/experiments/metrics.py eksportuje:

- temporal_split(test_ratio=0.2) — gwarancja że test = ostatnie X% czasu
- precision_at_k, recall_at_k, ndcg_at_k, map_score, coverage, diversity
- evaluate_method(method_callable, k=10) — uniwersalny wrapper

Plik backend/home/experiments/runner.py — klasa ExperimentRunner:

1. Wywołuje metodę rekomendacyjną dla każdego użytkownika z testu
2. Liczy metryki
3. Zapisuje wynik do tabeli ExperimentRun
4. Generuje raport CSV w experiments/results/

2.2. Duże modele językowe (LLM) i RAG — jak to działa w *SmartRecommender*

Poniżej jeden spójny opis techniczny, który w **pracy magisterskiej** można odwołać z tematów D3, D9, D11 i P4: co jest porównywane, jak przebiega inferencja i jakie biblioteki są sensowne w Pythonie.

2.2.1. Typowe dwa tryby użycia LLM w rekomendacji

(A) Re-ranking na liście kandydatów (*candidate generation + LLM*): najpierw tanio generujecie listę kandydatów N produktów (np. $N = 50-200$) — popularność, losowanie spójne z kategorią, opcjonalnie TF-IDF/CBF z istniejącego kodu lub proste k NN na embeddingach. Do LLM trafia **zwięźzony** kontekst: skrót historii zakupów / ostatnie oglądane, tytuły i krótkie opisy kandydatów, czasem agregat sentymentu. Model zwraca ranking lub permutację — **to ogranicza halucynacje**, bo model „wybiera tylko z podanej listy” (mimo to wymagana jest walidacja `product_id`).

(B) RAG na katalogu: embeddingi opisów produktów (i ewentualnie fragmentów opinii) trafiają do bazy wektorowej; dla użytkownika budujecie zapytanie tekstowe z historii; retrieval zwraca top- k fragmentów; LLM na tej podstawie wskazuje produkty lub nadaje im wagi. Baseline do porównania: ten sam retriever + scoring bez LLM, popularność, losowość z ograniczeniem kategorii — **nie musicie** porównywać z pełnym CF z inżynierki, jeśli pytanie badawcze dotyczy jakości RAG.

2.2.2. Co konkretnie porównujecie (przykładowe zestawy eksperymentów)

- **D3**: re-ranking (A) z różnymi promptami lub modelami vs. brak LLM (samo przycinanie listy kandydatów) vs. ewentualnie jeden wybrany kanał z repozytorium.
- **D9**: RAG (B) vs. retrieval + prosta reguła (np. średnia ocena) vs. popularność.
- **D11**: ten sam LLM z/walidacją `pydantic` i retry vs. „goły” tekst — metryka: odsetek odrzuconych odpowiedzi, czas, bezpieczeństwo.
- **P4**: jakość integracji: timeouty, kolejka, limity tokenów — często porównanie latencji i błędów między lokalnym Ollama a API (opcjonalnie).

2.2.3. Biblioteki i usługi (Python) — orientacyjny stos

Warstwa	Typowe pakiety / narzędzia
Inferencja lokalna	Ollama + klient <code>ollama</code> (Python) lub <code>httpx/requests</code> pod REST <code>localhost:11434</code>
API zewnętrzne (opcjonalnie, nie w osi pracy Olko)	<code>openai</code> , <code>litellm</code> , OpenRouter — płatne ; można pominąć, jeśli całość idzie lokalnie (Ollama na laptopie autora)
Embeddings	<code>sentence-transformers</code> , <code>torch</code> ; modele wielojęzyczne np. <code>paraphrase-multilingual-mpnet-base-v2</code>
Baza wektorowa	<code>qdrant-client</code> , <code>chromadb</code> , lub pgvector w PostgreSQL (jedna usługa mniej w Dockerze)
Orchestracja (opcjonalnie)	<code>langchain</code> , <code>llama-index</code> — wygodne, lecz niekonieczne na obronę
Walidacja wyjścia	Pydantic v2 (struktura JSON), ewent. <code>instructor</code> lub JSON schema w promptcie
Logowanie / metryki	istniejący <code>ExperimentRun</code> ; w kodzie <code>structlog</code> lub standard <code>logging</code>

Tabela 4: Stos technologiczny LLM/RAG w kontekście Django (orientacyjnie).

2.2.4. Czy da się “za darmo”?

Tak, w sensie licencji i braku klucza API dla modeli open-weight: modele takie jak Bielik, Mistral, Qwen w wariantach Apache 2.0 / podobnych można uruchomić lokalnie (**Ollama** — także natywnie na Apple Silicon, np. MacBook Pro M3 Pro z unified memory). **W praktyce na M3 Pro** sensowny punkt startu to Bielik/Mistral w rozmiarze 7B–8B (szybciej niż CPU-only PC); większe modele mogą wymagać dłuższego czasu lub zwężenia eksperymentu (mniejszy podzbiór użytkowników testowych, mniejsze N kandydatów do re-ranku). Warstwa embedding + Qdrant/pgvector jest darmowa w sensie oprogramowania. **W tej pracy przyjęto założenie autora:** oś eksperymentu bez API komercyjnych (GPT-4 itd.) — wyłącznie lokalne modele otwarte, dla reprodukowalności, zerowych kosztów tokenów oraz merytorycznego wykorzystania polskiego Bielika.

2.2.5. Przepływ żądania (uproszczony)

1. Pobranie historii użytkownika z ORM Django (zamówienia, interakcje).
2. Generacja kandydatów (A) lub retrieval (B).
3. Budowa promptu z twardym formatem wyjścia (JSON: lista {product_id, score}).
4. Wywołanie LLM; parsowanie; odrzucenie ID spoza bazy; ewentualnie druga próba z krótszym kontekstem.
5. Zapis wyniku ewaluacji do ExperimentRun / wyświetlenie w panelu admina (sekcja 2.3).

2.3. Panel badań w Django Admin — wyniki, uruchamianie eksperymentów, dobór metody

Sens w aplikacji: praca magisterska bazuje na systemie „prawdziwym”, więc badania nie powinny żyć wyłącznie w terminalu. Panel administratora (który już macie w Django) naturalnie agreguje dane i uprawnia — warto dodać **dedykowaną sekcję badawczą**: lista eksperymentów, porównanie metryk, możliwość ustawienia „aktywnej strategii” rekomendacji lub automatycznego wyboru wg ostatnich wyników.

2.3.1. Co rejestrujecie w adminie

- ExperimentRun — każdy wiersz to jeden uruchomiony eksperyment: metoda/kombinacja, parametry JSON, NDCG@10, Precision@10, czas, wersja seed_research. Lista z sortowaniem i filtrami (po dacie, po metodzie).
- UserPersona (opcjonalnie read-only) — podgląd, który użytkownik testowy należy do której osoby (debug).
- **Nowy model konfiguracyjny** (propozycja): np. ActiveRecommendationPolicy z polami: mode $\in \{fixed, auto_by_seed\}$, fixed_method_key, opcjonalnie JSON mapujący persona_name $\rightarrow$ method_key z ostatniego najlepszego ExperimentRun.

2.3.2. GUI zamiast “tylko manage.py”

Minimalny sensowny zestaw:

- strona listy ExperimentRun z “najlepszym runem” wyróżnionym (np. admin action lub adnotacja w notes);
- widok tabelaryczny (custom ModelAdmin z change_list_template lub osobny admin.view) porównujący zaznaczone runy obok siebie;
- przycisk „Uruchom ponownie z tymi parametrami” wywołujący ten sam kod co ExperimentRunner (w tle **Celery** lub synchronicznie z komunikatem „to może trwać X min”).

Dzięki temu promotor lub autor przy obronie mogą **zobaczyć** w przeglądarce, że wynik nie jest „wymyślony” — jest powiązany z rekordem w bazie i timestampem.

2.3.3. Zmiana metody w runtime — “auto” wg najlepszego wyniku

Idea: po serii eksperymentów offline wybieracie metodę M^* z najwyższym NDCG@10 (przy ustalonym seedzie i splicie). W bazie zapisujecie albo (i) globalnie jedną flagę „używaj M^* w produkcji/demo”, albo (ii) mapę per persona/segment: dla segmentu s metoda M_s^* , wyliczona z osobnej tabeli wyników (związek z tematem P2).

Warstwa API rekomendacji (np. wybrane views) na początku czyta ActiveRecommendationPolicy:

- **tryb** fixed — zawsze np. „CF+RRF” lub „LLM re-rank”;
- **tryb** auto_global — identyfikator metody z ostatniego “zwycięskiego” ExperimentRun oznaczonego w panelu;
- **tryb** auto_by_segment — na podstawie UserPersona lub klastra wybierana jest strategia z mapy (np. dla “cold” użytkowników LLM, dla “bogatej historii” fuzja).

Uzasadnienie merytoryczne: w pracy magisterskiej można opisać to jako *łączenie oceny offline z regułą wdrożeniową* — różne metody wygrywają w różnych sytuacjach (np. cold start vs. długa historia), więc sztywny jeden algorytm w kodzie produkcyjnym jest gorszy niż konfigurowalna polityka sterowana wynikami eksperymentów.

2.3.4. Nowa “zakładka” w adminie

Technicznie: osobna grupa modeli w admin.py z etykietą Research / Eksperymenty, ewentualnie **custom AdminSite** lub strona pod ścieżką /admin/research/dashboard/ z linkiem z głównego indeksu admina — by wyglądało jak „nowa zakładka” dla użytkownika końcowego administracji.

3. Katalog propozycji tematów magisterskich

Poniżej — **pula tematów** do uzgodnienia z promotorem. Nie trzeba realizować wszystkich wpisów; typowo jedna główna hipoteza + podrozdział (np. tuning lub skalowalność). **Warunek pracy nie jest** koniecznym porównywaniem z któryś z metod z inżynierki — liczy się merytoryczna rozbudowa istniejącego systemu (back + front tylko tam, gdzie trzeba). Stary kod bywa infrastrukturą lub *jednym* z baseline'ów; LLM, RAG, nowe modele lub tooling mogą stanowić główny wkład bez „pojedyńku” z konkretnym wido-kiem z inżynierki.

W tabelach 5 i 6 jest **pełna lista numerów** (D1–D12, P1–P12), żeby w PDF od razu widać było wiele opcji; poniżej — rozwinięcia. Kolumna **Inż.?**: ✓ = naturalne odniesienie do kodu inżynierskiego; ~ = można, ale nie musi; — = temat głównie samodzielny (nowy tor).

3.1. Szybki przegląd — pełna lista propozycji

Tabela 5: Dawid Olko — tematy D1–D12.

ID	Temat (skrót)	Inż.?
D1	Fuzja (CF + Apriori + sentyment) vs. metody pojedyncze	✓
D2	Tuning: CF, Apriori, sentyment (jakość vs. czas)	✓
D3	LLM jako ranker/re-ranker; odniesienie do klasycznych <i>opcjonalne</i>	~
D4	Rekomendacje z danych jawnych vs. implicit (z kolegą)	✓
D5	Skalowalność CF i reguł asocjacyjnych	✓
D6	seed_research, persony, manifest, powtarzalność	—
D7	Cold start użytkownika/produktu	✓
D8	Pipeline ewaluacji offline (ExperimentRun, CSV, metryki)	—
D9	LLM + RAG nad opisami produktów; baseline np. popularność	—
D10	Wyjaśnialność wybranego rankera (np. SHAP/LIME)	~
D11	Hardening LLM w backendzie: walidacja JSON, anty-iniekcja, katalog	—
D12	Bias/fairness rankingu między personami	—

Tabela 6: Piotr Smoła — tematy P1–P12.

ID	Temat (skrót)	Inż.?
P1	Sieć łącząca kanały score + kontekst; SHAP	✓
P2	Segmentacja; strategia rekomendacji per segment	✓
P3	Tuning CBF/TF-IDF, fuzzy, probabilistyka	✓
P4	Integracja LLM jako usługa (limity, walidacja ID)	—
P5	Profil z ukrytych interakcji (+ dwell) vs. jawny	✓
P6	Skalowalność CBF, fuzzy, ścieżek probabilistycznych	✓
P7	IT2-FLS vs. Mamdani T1	✓
P8	Meta-learner: boosting vs. MLP vs. fuzja	✓
P9	Ranker na embeddingach tekstu (k NN); bez obowiązku TF-IDF z inżynierki	—
P10	Graf współwystąpień produktów z koszyków	—
P11	Reprodukowalność: CI, skrypty, wersjonowanie wyników eksperymentów	—
P12	Dryf jakości / monitoring przy zmianie rozkładu danych	—

3.2. Dawid Olko — propozycje szczegółowe

3.3. Twój kod z inżynierki — gdzie co masz (mapa pod obronę)

W inżynierce zrobiłeś trzy główne „silniki”, z których korzysta sklep. W magisterce **nie musisz ich przepisywać** — musisz umieć wskazać plik i powiedzieć, że dokładasz **warstwę eksperymentów** (metryki + podpinanie istniejących algorytmów pod jeden interfejs).

Jedna wspólna myśl na obronie: klient w sklepie dalej widzi React + istniejące API; Ty w magisterce budujesz **ścieżkę badawczą**: ten sam kod rekomendacji wołany jest wielokrotnie na użytkownikach z `seed_research`, a wynik trafia do `ExperimentRun` i ewentualnie do zakładki w Django Admin (sekcja 2.3).

3.3.1. Łączenie metod rekomendacji versus metody pojedyncze (D1)

Cel i sens. Sprawdzić, czy w systemie e-commerce opartym o kod z inżynierki **łączenie rankingów** z kilku źródeł (CF, Apriori, kanał opinii/sentymetu) daje istotny zysk względem najlepszej pojedynczej metody — odpowiedź praktyczna dla małych i średnich sklepów z wieloma heurystykami.

Pytanie badawcze. Czy wybrane strategie fuzji (np. reciprocal rank fusion, głosowanie Bordy, ważona kombinacja znormalizowanych score) poprawiają $NDCG@K$ i $Precision@K$ w porównaniu z każdym rankerem osobno i z baseline popularności, przy stałym splitcie czasowym?

Dane, porównania. `seed_research` (persony, powtarzalny seed), jeden manifest wersji danych; baseline: singletony z inżynierki + popularność + ewentualnie ranking losowy.

Co mierzymy. $NDCG@K$, $Precision@K$, $Recall@K$, MAP, *coverage* katalogu, ewentualnie *diversity*; czas odpowiedzi na użytkownika.

Promotor. Kombinacja metod kontra metody pojedyncze.

Jak to powiedzieć na obronie (bez żargonu). Masz trzy **niezależne listy** rekomendacji dla tego samego użytkownika: (1) „co podobne do tego co kupiłeś” (CF), (2) „co ludzie dokupują do Twojego koszyka” (Apriori), (3) „co ma dobre opinie w tonacji podobnej do Ciebie” (sentymet). Fuzja to po prostu **łączenie trzech list w jedną** według reguły (np. RRF — każda metoda głosuje na produkty). Pytanie badawcze: czy taka „średnia” jest lepsza niż jedna wybrana metoda i niż „najpopularniejsze w sklepie”.

Konkretnie w Twoim repozytorium. Nie ruszasz na siłę widoków klienta — piszesz np. `home/experiments/recommender` wołające te same zapytania/logikę co CF (macierz z `OrderProduct`, podobieństwa z `ProductSimilarity`), osobno wrapper na reguły z `ProductAssociation`, osobno ranking z sentymentu (logika z `CustomSentimentAnalysis`). Potem `home/experiments/fusion.py` łączy wyniki. Ewaluacja: dla użytkowników z **ostatniego okna czasu** (test) sprawdzasz, czy produkty, które faktycznie kupili, pojawiają się w $top-K$ Twojej listy — to są $Precision / NDCG$ z `metrics.py`.

3.3.2. Wpływ hiperparametrów na jakość i czas — CF, Apriori, sentyment (D2)

Cel i sens. Udokumentować **kompromis jakość–czas obliczeniowy** dla metod po stronie Olko: które parametry realnie przesuują ranking (np. próg podobieństwa CF, *min_support* / minimum pewności reguł, agregacja sentymentu do poziomu produktu).

Pytanie badawcze. Które hiperparametry są najbardziej czułe na jakość rekomendacji przy ustalonym protokole testowym, a które tylko kosztują czas przeliczenia?

Dane, porównania. Ta sama baza co w całym zespole; siatka parametrów lub Optuna; porównanie z domyślnymi ustawieniami z inżynierki.

Co mierzymy. Te same metryki rankingowe co w całej pracy; dodatkowo czas budowy macierzy podobieństwa / reguł asocjacyjnych, licznosc zapisanych reguł — wykresy Pareto (jakość vs. czas).

Promotor. Hyperparameter tuning (promotorska lista).

Jak to powiedzieć na obronie. **Hiperparametr** to “pokrętko” w algorytmie: np. jak **często** para produktów musi wystąpić w zamówieniach, żeby uznać regułę (to jest *min_support* w Apriori), albo od jakiego **progu podobieństwa** zapisujesz parę produktów w CF (w kodzie jest *m.in.* próg przy budowie list podobieństwa). Tuning = systematyczna zmiana tych pokręteł i pomiar: czy ranking się poprawia, a ile rośnie czas liczenia / liczba reguł w bazie.

Konkretnie w Twoim repozytorium. Apriori: klasa `CustomAssociationRules(_, min_support, min_confidence)` oraz endpoint aktualizacji reguł przyjmujący *min_support* z JSON (`association_views.py`) — te wartości możesz zamienić w pętli eksperymentów. CF: w `recommendation_views.py` przy generowaniu podobieństw jest `m.in. similarity_threshold = 0.5` (i wcześniej w CBF w `custom_recommendation_engine.py` jest `similarity_threshold = 0.2` — to osobna metoda Piotra, ale pokazuje schemat). Sentyment: możesz stroić progi “positive/negative” lub sposób agregacji opinii do jednej liczby na produkt. Wszystko zapisujesz w `ExperimentRun.parameters_json` + metryki.

3.3.3. Duży model językowy jako główny kanał rekomendacji (D3; porównanie z inżynierką opcjonalne)

Cel i sens. Zbadać jakość rekomendacji, gdy **LLM** (lokalny lub API) jest głównym źródłem rankingu lub **re-rankingu** na zwężonej liście kandydatów (tryb (A) w sekcji 2.2). Praca może — lecz nie musi — zawierać porównanie z najlepszą heurystyką z kodu inżynierskiego; dozwolone jest użycie prostszego baseline (popularność, losowość z ograniczeniem kategorii, ostatnio oglądane), jeśli pytanie badawcze dotyczy np. stabilności promptu, kosztu czasu lub halucynacji.

Jak to działa w systemie (krok po kroku).

1. Generacja **kandydatów** N (np. 80–150): tania metoda bez LLM — żeby nie wpychać całego katalogu do kontekstu.

2. Z ORM składany jest **kontekst użytkownika**: skrócona historia zakupów, ewentualnie średnie sentymenty lub cytaty z opinii (ograniczone liczbą tokenów).
3. LLM dostaje instrukcję: zwrócić wyłącznie JSON z listą `product_id` z kandydatów + ranking/score.
4. **Walidacja**: odrzucenie nieistniejących ID; ewentualnie drugie wywołanie przy błędzie składni (patrz D11).
5. Wynik oceniany jest **tym samym** modulem metryk co pozostałe metody (`seed_research`, split czasowy).

Co porównujecie (propozycja zestawu).

- kilka modeli LLM (np. Bielik 7B vs. 11B vs. Mistral) przy tym samym prompcie;
- zero-shot vs. few-shot (2–3 przykłady „dobrych” list w prompcie);
- LLM re-rank vs. ten sam zestaw kandydatów bez LLM (np. sortowanie po score z generatora);
- opcjonalnie: jedna wybrana metoda z repozytorium jako dodatkowy baseline — jeśli promotor tego wymaga.

Biblioteki, koszt „zero API”. Szczegółowo: tabela w sekcji 2.2 (`ollama/httpx`, `pydantic`, ewent. `sentence-transformers`).
Uruchomienie lokalne = brak opłat za tokeny; w pracy opisujecie zużycie RAM/VRAM i czas.

Powiązanie z GUI. Każdy skonfigurowany wariant (model + prompt + N) zapisujecie jako osobny lub wersjonowane `ExperimentRun`; w panelu admina (sekcja 2.3) możecie oznaczyć zwycięski run i — jeśli implementujecie politykę — podpiąć go pod tryb produkcyjny/demo.

Pytanie badawcze. Przy ustalonym protokole (split czasowy na `seed_research`) — czy wybrana strategia LLM osiąga akceptowalny $NDCG@K$ i jak wygląda stabilność vs. halucynacje?

Dane, porównania. `seed_research`; baseline minimum: brak LLM na tych samych kandydatach; dalsze baseline do uzgodnienia z promotorem.

Co mierzymy. $NDCG@K$, $Precision@K$; odsetek odrzuconych / poprawionych odpowiedzi; latency na jedno wywołanie; ewentualnie koszt tokenów przy API (plan B).

Promotor. LLM jako kierunek akceptowalny przez promotora; doprecyzowanie porównań na spotkaniu.

Jak to powiedzieć na obronie. LLM tutaj **nie zna całego katalogu z pamięci**. Najpierw system (Twój kod w Django) buduje **krótką listę kandydatów** (np. 100 produktów) prostszą metodą; potem model tylko **przestawia** lub wybiera z tej listy na podstawie opisów i historii. Dzięki temu łatwiej wytłumaczyć: model sugeruje kolejność „z tego co już wybraliśmy algorytmem”, a Ty mierzysz, czy ta kolejność jest lepsza na danych testowych. Porównanie z CF/Apriori jest **możliwe**, ale nie jest obowiązkowe — możesz porównać się z popularnością.

Konkretnie w Twoim repozytorium. Nowy moduł np. `home/experiments/llm_recommender.py`: pobiera zamówienia z `Order/OrderProduct`, kandydatów np. z `ProductAssociation` lub najpopularniejszych w kategorii użytkownika, buduje prompt, woła Ollamę (`httpx` lub pakiet `ollama`), parsuje JSON. Istniejące endpointy CF/sentymentu zostają; ewaluacja idzie tym samym `runner.py` co reszta. Na obronie pokazujesz też konfigurację modelu zapisane w `ExperimentRun`.

3.3.4. Jakość rekomendacji przy profilu z danych jawnych (opinie, oceny) (D4)

Cel i sens. Zbudować rankery wykorzystujące wyłącznie **jawny** sygnał (opinie, oceny, zagregowany sentyment) i porównać je z modelem opartym o interakcje ukryte — na **tym samym** zadaniu przewidywania (np. następny zakup w obszarze testowym).

Pytanie badawcze. Czy przy dostępnych opiniach profil jawny wystarcza do kompetitivenego rankingu względem profilu z samych interakcji (realizowanego równolegle przez kolegę)?

Dane, porównania. `seed_research` z realistycznym powiązaniem opinii z produktami; druga gałąź (implicit) eksperymentalnie wyłączana lub realizowana przez Smołę — wspólna tabela wyników.

Co mierzymy. Metryki rankingowe na zdarzeniach zakupu w teście; ewentualnie błąd regresji oceny — jeśli ustalicie taki dodatkowy eksperyment.

Promotor. Jawne vs ukryte preferencje.

Jak to powiedzieć na obronie. **Jawne** = to, co użytkownik **napisał lub ocenił** (tekst opinii, gwiazdki). **Ukryte (implicit)** = to, co robi w aplikacji bez opinii (kliknięcia, czas na karcie — u Ciebie głównie robi to Piotr). Temat D4: budujesz rekomendację **tylko z opinii/ocen** i porównujesz z jego modelem na interakcjach — **na tym samym zadaniu** “co kupi w przyszłości w zbiorze testowym”. Sens: sprawdzić, czy warto zbierać opinie, czy wystarczą ślady zachowania.

Konkretnie w Twoim repozytorium. Dane: modele `Opinion` (+ oceny) w Django; sentyment liczy `CustomSentimentAnal`. Ty definiujesz **jedną ustaloną regułę**: np. `score produktu = średni sentyment opinii minus kara za mało recenzji`, albo ranking tylko z produktów skomentowanych przez użytkownika. Nie musisz zmieniać Reacta — całość w backendzie + eksperyment. Kolega liczy swój ranking z `UserInteraction`; oboje zapisujecie wyniki w `ExperimentRun`.

3.3.5. Skalowalność collaborative filtering i reguł asocjacyjnych (D5)

Cel i sens. Pokazać, jak rosnąca skala $|U|$, $|I|$ i liczby transakcji wpływa na **czas przeliczenia i zużycie pamięci** dla CF i Apriori oraz na rozmiar tabel pośrednich w PostgreSQL — z identyfikacją wąskich gardeł zapytań.

Pytanie badawcze. Przy której skali prosty pipeline przestaje być akceptowalny czasowo i jakie techniki (indeksy, batch, cache) dają mierzalną poprawę?

Dane, porównania. Seria konfiguracji `seed_research` (np. N użytkowników rosnące z krokiem); te same metryki jakości na jednym podzbiorze referencyjnym.

Co mierzymy. Czas budowy modelu, czas pojedynczej rekomendacji, rozmiar `ProductSimilarity` / tabel reguł; EXPLAIN dla najcięższych zapytań; jakość vs. skala (czy spada przy rzadszych danych).

Promotor. Skalowalność.

Jak to powiedzieć na obronie. Skalowalność = co się dzieje, gdy rośnie liczba **użytkowników, produktów i zamówień**; czy przeliczenie podobieństw CF i reguł Apriori nadąża, czy baza rośnie do gigantycznych rozmiarów, które zapytania są najwolniejsze. To typowy rozdział „inżynierski” obok metryk jakości.

Konkretnie w Twoim repozytorium. Mierzysz czas regeneracji macierzy CF (logika w `recommendation_views.py` przy aktualizacji `ProductSimilarity`) i czas `update-association-rules` na większej liczbie transakcji z `seed_research`. Patrzysz na rozmiar tabel `ProductSimilarity` (CF) i `ProductAssociation`. Opcjonalnie uruchamiasz EXPLAIN na najcięższych zapytaniach ORM. Nie musisz zmieniać algorytmu na siłę — możesz opisać progi (cache, batch, indeksy), przy których system jest jeszcze akceptowalny.

3.3.6. Kontrolowany, powtarzalny zbiór danych pod ewaluację metod (D6)

Cel i sens. Uczynić **głównym wkładem** projekt `person`, manifestu wersji danych i komendy `seed_research`, tak by każda praca w zespole cytowała ten sam protokół — dokument metodyczny + implementacja.

Pytanie badawcze. Czy zaprojektowany zbiór odtwarza zjawiska istotne dla metod z inżynierki (np. długi ogon, sezonowość, nierówny udział `person`) i czy wyniki eksperymentów są stabilne między powtórzeniami seeda?

Dane, porównania. Opis `person` vs. statystyki wygenerowanej bazy; testy stabilności metryk przy tym samym *random seed*.

Co mierzymy. Rozkłady interakcji, pokrycie kategorii, licznosc cold start; wariancja wybranych metryk między runami.

Promotor. Ogólne — podstawa pod każdy punkt z listy promotora (porównywalne eksperymenty).

Jak to powiedzieć na obronie. Teraz po odpaleniu zwykłego `seed.py` dostajesz małą, dość losową bazę — na obronie trudno bronić liczb „bo tak”. `seed_research` to **sztucznie zaprojektowany**, powtarzalny zestaw danych (`persony`, spójne koszyki, opinie po zakupie), żebyś mógł powiedzieć: „oto wersja danych 1.0, seed 42, każdy może to odtworzyć”.

Konkretnie w Twoim repozytorium. Nowa komenda `manage.py seed_research` (osobny plik obok `seed.py` — nie kasujesz starego). Zapisujesz wersję w manifestcie i w polu `seed_research_version` przy `ExperimentRun`. W pracy opisujesz `persony` i statystyki wygenerowanej bazy (ile zamówień, jak rozłożone kategorie). To może być **główny wkład** inżynierski, nawet jeśli algorytm jest prosty.

3.3.7. Cold start użytkownika lub produktu (D7)

Cel i sens. Zbadać zachowanie metod Olko przy **sztucznie obciętej historii** lub nowych kontaktach: kiedy klasyczny CF zawodzi, a sentyment lub reguły asocjacyjne utrzymują jakość.

Pytanie badawcze. Przy jakiej liczbie interakcji lub zamówień CF osiąga próg 'używalności', i czy hybryda z sentymentem/Apriori redukuje spadek $NDCG@K$?

Dane, porównania. Warianty `seed_research` z ograniczoną historią per persona; pełny model jako górne odniesienie.

Co mierzymy. $NDCG@K$, $Precision@K$ w funkcji długości historii; ewentualnie odsetek użytkowników bez rekomendacji CF.

Promotor. Praktyczne uzupełnienie tematu kombinacji metod i LLM (łagodzenie cold start).

Jak to powiedzieć na obronie. **Cold start** = nowy użytkownik lub produkt bez historii; CF często wtedy **nic sensownego nie ma z czego policzyć**. Pytanie: kiedy zaczyna działać CF, a kiedy ratują np. reguły koszykowe (bo działają globalnie) albo sentyment z opinii przy produkcie.

Konkretnie w Twoim repozytorium. W `seed_research` generujesz warianty z **obciętą historią** (np. tylko 1 zamówienie) vs. pełna historia; dla produktu — symulujesz "nowy" przedmiot bez podobieństw w `ProductSimilarity`. Uruchamiasz te same funkcje co dziś (CF z widoku, reguły z `ProductAssociation`, sentyment). Pokazujesz wykres: oś X = ile interakcji, oś Y = $NDCG@10$.

3.3.8. Jednolity pipeline ewaluacji offline jako element pracy (D8)

Cel i sens. Sformalizować **narzędzie badawcze**: split czasowy, moduł metryk, zapis `ExperimentRun` / CSV — i na nim wykonać 2–3 konkretne porównania hipotez (np. fuzja vs. singleton, jeden zestaw strojów).

Pytanie badawcze. Czy zaprojektowany pipeline jest wystarczający do reprodukowalnych wyników w całym zespole i jak wygląda niepewność wyników przy bootstrapie po użytkownikach?

Dane, porównania. Manifest + wersjonowanie kodu eksperymentu; porównanie z ad-hoc logowaniem z konsoli (jeśli było).

Co mierzymy. Zgodność wyników między powtórzeniami; dokumentacja czasu przeliczenia; metryki dla zdefiniowanych hipotez pomocniczych.

Promotor. Infrastruktura pod wszystkie punkty listy (metryki, porównania).

Jak to powiedzieć na obronie. **Pipeline** = zawsze ten sam przepis: (1) wczytaj dane po `seed_research`, (2) podziel czasowo na trening/test, (3) dla każdego użytkownika w teście poproś metodę o listę $\text{top-}K$, (4) sprawdź, czy faktyczne przyszłe zakupy są w tej liście, (5) uśrednij — to jest Precision/NDCG. Dzięki temu każdy eksperyment jest **porównywalny**.

Konkretnie w Twoim repozytorium. Implementacja: `home/experiments/metrics.py + runner.py`; wynik w `ExperimentRun` i CSV. W adminie rejestrujesz model (sekcja 2.3). Na pytanie „czy wynik jest wiarygodny” odpowiadasz: „tak, bo ten sam kod liczy metryki dla wszystkich metod i zapisuje parametry w JSON”.

3.3.9. LLM i RAG wyłącznie nad treścią katalogu (D9)

Cel i sens. Zbudować tor rekomendacji, w którym **semantic search + LLM** na opisach produktów, atrybutach i ewentualnie skrótach opinii dominuje nad heurystyką transakcyjną (tryb (B) w sekcji 2.2). Metody z inżynierki **nie są wymagane**; typowy baseline to popularność, losowość z boundingiem kategorii lub **ten sam retriever** (np. $\text{top-}k$ embeddingów) + prosta funkcja łącząca **bez** wywołania LLM — żeby pokazać, co dokładnie dokłada warstwa generatywna.

Implementacja RAG (szczegóły).

1. **Indeks:** dla każdego produktu łączyć pole tekstowe (nazwa + opis + kategoria); liczyć embedding (np. `sentence-transformers`); zapis do Qdrant/Chroma/**pgvector**.
2. **Zapytanie:** z historii zakupów/użytkownika budujecie tekst zapytania (“użytkownik kupuje sprzęt gamingowy, lubi niską latencję myszek”).
3. **Retrieval:** k najbliższych produktów; opcjonalnie MMR żeby ograniczyć duplikaty w jednej kategorii.
4. **LLM:** na podstawie skróconych opisów retrieved produktów zwraca finalną listę rankingową lub odrzuca niepasujące — zawsze z walidacją `product_id`.

Porównania. RAG+LLM vs. sam retrieval (np. średnia ocena wśród retrieved) vs. popularność globalna; opcjonalnie vs. jeden prosty kanał z repo — jeśli potrzebny dodatkowy cytat w rozdziale porównawczym.

Biblioteki i koszt. Jak w sekcji 2.2: `sentence-transformers`, klient bazy wektorowej, Ollama; **bez opłat** przy lokalnym inference.

Panel admina. Każda konfiguracja (model embeddingu, k , wersja indeksu) — osobny wpis w `ExperimentRun` lub powiązana tabela `EmbeddingIndexVersion`; podgląd w sekcji 2.3.

Pytanie badawcze. Czy RAG na lokalnym katalogu poprawia trafność w porównaniu z prostym baseline przy ograniczonym kontekście okna LLM?

Dane, porównania. `seed_research` (opinie i opisy spójne z produktami); ewaluacja na tym samym zadaniu co reszta zespołu (np. następny zakup w teście).

Co mierzymy. $NDCG@K$, $Precision@K$; czas indeksacji i czas zapytania end-to-end; ewentualnie jakość retrieval ($recall@k$ na zawartości koszyka testowego).

Promotor. LLM — naturalne rozszerzenie; dopasowanie brzmienia tytułu z promotorem.

Jak to powiedzieć na obronie. To jest inny tor niż CF: najpierw zamieniasz opisy produktów na **wektory** (embeddingi), szukasz podobnych do “profilu tekstowego” użytkownika, a LLM tylko pomaga ułożyć końcową listę z znalezionych fragmentów. Porównujesz: “sam retrieval” vs. “retrieval + LLM” — żeby pokazać, czy model językowy coś realnie dokłada.

Konkretnie w Twoim repozytorium. Źródłem tekstu są pola produktu w Django (Product: nazwa, opis, kategoria). Nowy skrypt indeksujący (poza requestem HTTP, wołany z management command): `sentence-transformers` → baza wektorowa lub `pgvector`. Nie musisz usuwać `CustomSentimentAnalysis` — możesz dodać embedding opinii do tekstu produktu. Wyniki eksperymentów jak zwykle w `ExperimentRun`.

3.3.10. Wyjaśnialność wybranego rankera (D10)

Cel i sens. Dla **jednego** ustalonego pipeline (np. wybranej fuzji, modelu z kolegi lub własnego rankera) pokazać, które cechy wejścia (produkty, kategorie, historia) najmocniej wpływają na pozycję w rankingu — metody typu SHAP/LIME lub analiza wrażliwości, zgodna z literaturą XAI.

Pytanie badawcze. Czy wyjaśnienia są stabilne dla podobnych użytkowników i czy zgadzają się z intencją person w `seed_research`?

Dane, porównania. Podzbiór użytkowników testowych; możliwe porównanie dwóch architektur wyjaśnień.

Co mierzymy. Jakość proxy wyjaśnień (np. fidelity), ewentualnie studia przypadków; metryki rankingu wtórne.

Promotor. Możliwe jako uzupełnienie pracy o LLM lub ensemble.

Jak to powiedzieć na obronie. Na obronie ktoś może zapytać: „dlaczego ten produkt jest na 1. miejscu?”. W tej pracy wybierasz **jeden** ustalony pipeline (np. Twoja fuzja D1) i pokazujesz, które cechy (historia, kategoria, reguła koszykowa) najbardziej przesunęły ranking — narzędzia typu SHAP/LIME albo prostsza analiza wrażliwości, zgodnie z literaturą XAI.

Konkretnie w Twoim repozytorium. Bierzesz wektory cech, które już liczysz wewnątrz eksperymentu (np. score z CF, liczba reguł wspierających produkt, średni sentyment). Pakietowo: `shap` przy prostym modelu albo wyjaśnienia dla surrogate modelu. Nie musisz dotykać React — w pracy pokazujesz wykresy + 2–3 przykłady użytkowników z `seed_research`.

3.3.11. Bezpieczeństwo i walidacja wyjść LLM w Django (D11)

Cel i sens. Opracować **warstwę wiarygodnego wyjścia**: twardy kontrakt (JSON według schematu Pydantic), odrzucanie ID spoza katalogu, limit długości promptu, eskejpowanie lub sanitacja fragmentów opinii użytkownika wpinanych do promptu (mitigacja *prompt injection*). Bez przepisywania całego frontu — logika w backendzie i ewentualnie testy jednostkowe szablonów.

Jak to jest zbudowane (technicznie).

- szablon promptu w jednym miejscu (np. `llm/prompts/recommend_v1.txt`) z symbolami zastępczymi;
- odpowiedź parsowana przez Pydantic; przy `ValidationError` — skrócenie kontekstu lub druga próba z instrukcją „popraw tylko JSON”;
- testy z **złośliwymi** wstawkami w polach opinii (zsyntetykowane w `seed_research` lub `fixture`) — pomiar odsetka przypadków, w których model próbuje wygenerować niedozwolone ID lub ignoruje format.

Pełniejsza lista narzędzi: sekcja 2.2.

Co porównujecie. Model A vs. B **pod kątem conformance** (czy przestrzegają JSON); strategia z retry vs. bez; oraz (opcjonalnie) wpływ walidacji na $NDCG@K$ (czasem filtr „uczciwy” obniża metrykę, ale rośnie bezpieczeństwo).

GUI / audyt. W Django Admin przy każdym `ExperimentRun` pole `notes` lub osobna tabela `LLMValidationAudit` z liczbą odrzuceń — żeby na obronie pokazać tabelę „błędów parsowania na 1000 wywołań” (sekcja 2.3).

Pytanie badawcze. Jak często modele X i Y łamią kontrakt wyjściowy i jakie wzorce ataku (np. iniekcja w tekście opinii) są skuteczne w Waszej architekturze?

Dane, porównania. `seed_research` + syntetyczne „złośliwe” fragmenty w kontrolowanym benchu.

Co mierzymy. Błędy walidacji (%), fallback rate, czas obsługi; ewentualnie skrótowny wpływ na metryki po filtracji.

Promotor. Nowoczesny kierunek (LLM + bezpieczeństwo) — do uzgodnienia.

Jak to powiedzieć na obronie. LLM zwraca tekst — tekst może być zły (zły JSON, wymyślone ID produktu, albo „instrukcja” podszyta pod opinię użytkownika). Ten temat to **twardy filtr w Django**: parsowanie Pydantic, odrzucanie ID spoza bazy, ewentualnie skrótny promptu i ponowienie. Na obronie: „pokazuję, jak często model psuje format i jak na to reagujemy”.

Konkretnie w Twoim repozytorium. Middleware/servis przy wywołaniu modelu w `llm_recommender.py`; testy na sztucznych opiniach w `seed_research`. Wynik audytu zapisujesz w `notes` lub osobnej tabeli; w adminie widać skalowanie błędów (sekcja 2.3).

3.3.12. Uczciwość rankingu w podgrupach person (D12)

Cel i sens. Sprawdzić, czy wybrany algorytm (niekoniecznie z inżynierki) **faworyzuje** niektóre persony (np. `office_worker` vs. `gamer`) pod względem $NDCG@K$ lub *exposure* w *top-K*.

Pytanie badawcze. Czy różnice między grupami są istotne i czy prosta korekta (np. równanie wag) je zmniejsza bez dużej utraty jakości globalnej?

Dane, porównania. `seed_research` z znanym przypisaniem `UserPersona`; porównanie przed/po korekcie.

Co mierzymy. Metryki per persona + agregaty (np. min-max spread); globalny $NDCG@K$.

Promotor. Opcjonalny, silny temat interdyscyplinarny — do akceptacji.

Jak to powiedzieć na obronie. Dzięki personom w `seed_research` wiesz, kto jest np. `gamer`, a kto `office_worker`. Pytanie: czy Twój algorytm nie faworyzuje jednej grupy (np. zawsze lepszy $NDCG$ dla `gamerów`)? To wygląda jak jakość „sprawiedliwości” rekomendacji, nie tylko średnia globalna.

Konkretnie w Twoim repozytorium. Tabela `UserPersona` (propozycja w tym dokumencie) łączy użytkownika z nazwą persony. Liczysz metryki **osobno** dla każdej persony, potem porównujesz z metryką globalną. Możesz zaproponować prostą korektę wag w fuzji (D1) per persona — i pokazać, czy minimalnie poprawiasz spread między grupami.

3.4. Piotr Smoła — propozycje szczegółowe

3.4.1. Sieć neuronowa łącząca wiele metod z kontekstem użytkownika (P1)

Cel i sens. Zastąpić ręczną fuzję **uczonym rankerem**: wektor normalizowanych score z dostępnych metod (w praktyce do 9 kanałów z inżynierki) plus cechy użytkownika (aktywność, entropia kategorii itd.); ocenić zysk względem najlepszej heurystyki i prostszego ensemble.

Pytanie badawcze. Czy MLP (lub podobna sieć) istotnie poprawia ranking w $NDCG@K$ i które metody dominują w których regionach przestrzeni cech — interpretacja przez SHAP lub analogiczną metodę.

Dane, porównania. Wspólny bootstrap / split; baseline: najlepszy singleton, RRF, ewentualnie boosting (por. osobna pozycja katalogu).

Co mierzymy. Metryki rankingowe; krzywe uczenia; analiza ważności cech; czas treningu i inferencji.

Promotor. Sieć łącząca metody + dominacja źródeł.

3.4.2. Segmentacja użytkowników (K-Means, DBSCAN) i strategia per segment (P2)

Cel i sens. Grupować użytkowników po cechach z zamówień i interakcji, następnie dobierać **wagi fuzji** lub **wybór metody** osobno w każdym segmencie — porównać z jednym globalnym ustawieniem.

Pytanie badawcze. Czy segmentacja redukuje błąd rankingowy “średnio po użytkownikach” i czy różne segmenty wymagają różnych wag (np. większy udział CBF w segmencie “nowi”)?

Dane, porównania. Wyniki globalne vs. per-segment; stabilność klastrow przy różnym K (silhouette, sensowność biznesowa person).

Co mierzymy. NDCG@ K globalnie i warunkowo na segment; liczba użytkowników w klastrach; czas klasteryzacji.

Promotor. Segmentacja + optymalna kombinacja per segment.

3.4.3. Hyperparameter tuning — CBF, silnik rozmyty, modele probabilistyczne (P3)

Cel i sens. Systematycznie zestroić parametry po stronie Smoły (TF-IDF / CBF, funkcje przynależności i reguły w `fuzzy_logic_engine`, parametry Markowa i NBC), pokazując wpływ na metryki i koszt obliczeń.

Pytanie badawcze. Które parametry dają największy przyrost jakości przy rozsądnym czasie, a które są “płaskie”?

Dane, porównania. Ustalony split; porównanie z wartościami domyślnymi z inżynierki; ewentualna wspólna tabela z pracy zespołowej jako załącznik.

Co mierzymy. NDCG@ K , inne metryki zespołu; czas przeliczenia CBF i ewaluacji reguł rozmytych.

Promotor. Tuning hiperparametrów.

3.4.4. Warstwa integracji LLM (API, walidacja, koszt, niezawodność) (P4)

Cel i sens. Potraktować LLM jako **usługę wewnętrzną** Django: jeden moduł (np. `home/llm/service.py`) odpowiedzialny za timeouty, ponawianie, limity równoległych zapytań, logowanie błędów i **wspólną walidację** odpowiedzi. Dzięki temu Olko może skupić się na hipotezach badawczych, a Ty — na niezawodności i obserwowalności — naturalny podział w zespole. Porównanie z metodami klasycznymi **nie jest konieczne**: możecie raportować jakość usługi (SLA) i koszt czasu względem lokalnego Ollama vs. ewentualnego API.

Architektura (propozycja).

- **Adapter:** `LLMBackend` z implementacjami `OllamaBackend`, `OpenAICompatibleBackend` — ten sam interfejs `complete(prompt) -> str`.
- **Polityka:** kolejka (np. `django-rq` / Celery) na długie batch-e eksperymentów; krótkie żądania synchronicznie z timeoutem.
- **Metryki operacyjne** (opcjonalnie, w stylu Prometheus): licznik błędów, histogram latency — lub zapis do `ExperimentRun.notes`.

Biblioteki: sekcja 2.2.

Co porównujecie / badacie. Lokalny Ollama vs. API (jeśli macie klucz testowy); różne limity `max_tokens`; strategia 1 wywołanie vs. „skrót kontekst i powtórz”; wpływ tych decyzji na **udział poprawnie zwalidowanych list** rekomendacji.

Panel admina. Konfiguracja endpointu LLM, kluczy (przez `django-envirom`, nie w repo), przełącznik „środowisko lab vs prod” — formularz w adminie (`model LLMServiceConfig`) opisany w sekcji 2.3; link do listy `ExperimentRun` filtrowanej po `method_or_combination` zawierającym `llm`.

Pytanie badawcze. Przy tym samym module `evaluate` jak reszta systemu — jaki jest rozkład czasu odpowiedzi i odsetek odrzuconych odpowiedzi po walidacji przy różnych backendach?

Dane, porównania. Ten sam ground truth co u Olko, jeśli łączycie eksperymenty; równolegle możecie mieć tryb “dry-run” tylko na latency bez pełnej ewaluacji rankingowej.

Co mierzymy. Latency (p50, p95), throughput, error rate, metryki jakości rankingu po walidacji (gdy uruchamiacie pełny pipeline).

Promotor. LLM — strona systemowa / integracyjna; uzgodnienie brzmienia z promotorem.

3.4.5. Rekomendacje z danych ukrytych (interakcje, ewentualnie dwell) (P5)

Cel i sens. Zbadać profile zbudowane tylko z `UserInteraction` (z ewentualnym decay czasu i czasem na karcie produktu), **bez** opinii jawnych — na tym samym zadaniu co gałąź “jawna” Olko.

Pytanie badawcze. Czy implicit + ewentualnie dwell przewyższa lub dorównuje profilowi opartemu o opinie przy przewidywaniu zakupów w teście?

Dane, porównania. Wspólny seed z realistyczną ścieżką “oglądanie przed zakupem”; porównanie z zdefiniowaną przez kolegę gałęzią jawną.

Co mierzymy. Standardowe metryki rankingowe; ewentualnie AUC jeśli sformułujecie zadanie binarne “kupi / nie kupi”.

Promotor. Jawne vs ukryte — druga strona porównania.

3.4.6. Skalowalność CBF, logiki rozmytej i ścieżek probabilistycznych (P6)

Cel i sens. Ocenąć koszt przeliczenia TF-IDF po całym korpusie opisów, liczby reguł rozmytych i zapytań Markova przy rosnącej skali danych — inne wąskie gardła niż po stronie CF.

Pytanie badawcze. Które operacje dominują czas w PostgreSQL / Pythonie i jak skaluje się jakość vs. rozmiar korpusu?

Dane, porównania. Seria konfiguracji `seed_research` o rosnącej liczbie użytkowników, produktów i transakcji; na wybranych punktach skali — identyczny (lub proporcjonalny) protokół testowy, by oddzielić wpływ rozmiaru danych od przypadkowej próby.

Co mierzymy. Czas budowy wektorów, czas jednej rekomendacji, rozmiar struktur pośrednich; metryki jakości na stałym podzbiorze testowym.

Promotor. Skalowalność.

3.4.7. Wnioskowanie rozmyte typu drugiego (IT2-FLS) wobec Mamdani T1 z inżynierki (P7)

Cel i sens. Wprowadzić **niepewność** do wejść rozmytych (np. szerzej interpretowane sentymenty lub rzadsze opinie jako przedziały) i porównać system IT2 (np. biblioteka `pyIT2FLS`) z obecną logiką T1 pod kątem jakości rankingu i interpretowalności.

Pytanie badawcze. Czy IT2-FLS poprawia lub stabilizuje wyniki przy szumnych danych wejściowych względem T1 przy tym samym protokole testowym?

Dane, porównania. Mapowanie cech do przedziałów przynależności; baseline: silnik z `fuzzy_logic_engine`.

Co mierzymy. $NDCG@K$ i pokrewne; ewentualnie sensowność defuzyfikacji (spójność z porządkiem preferencji); czas inferencji IT2 vs. T1.

Promotor. Wnioskowanie przedziałowe — propozycja promotora.

3.4.8. Meta-uczenie prostsze od głębokiej sieci (np. gradient boosting) (P8)

Cel i sens. Sprawdzić, czy **gradient boosting** na wektorze score z metod + kontekst daje jakość bliską MLP przy mniejszej złożoności i łatwiejszej interpretacji częściowej.

Pytanie badawcze. Czy LightGBM / XGBoost na zgrubnie zdefiniowanych cechach nie gorsi znacząco sieci i jak wygląda mapa ważności cech vs. SHAP dla MLP?

Dane, porównania. Te same wektory co dla sieci łączącej; porównanie z MLP i prostą fuzją.

Co mierzymy. $NDCG@K$, czas treningu, rozmiar modelu, interpretacja ważności.

Promotor. Sieć łącząca — alternatywa lżejsza i dobra do dyskusji o dominacji metod.

3.4.9. Ranker na embeddingach tekstu produktów (P9)

Cel i sens. Wdrożyć **nowy kanał** rekomendacji: embeddingi opisów/nazw (np. modele `sentence-transformers`), podobieństwo w przestrzeni wektorowej, ranking k najbliższych sąsiadów względem profilu użytkownika złożonego z historii. **Nie wymaga** porównania z TF-IDF z inżynierki — można potraktować CBF z inżynierki jako opcjonalny baseline.

Pytanie badawcze. Czy jakość rankingu i czas inferencji są akceptowalne przy rozmiarze katalogu z `seed_research`?

Dane, porównania. Ten sam split i metryki co zespół; baseline: popularność lub prosty k NN na TF-IDF (świadomie dodany pod pracę).

Co mierzymy. $NDCG@K$, czas budowy indeksu (FAISS/opcja czysto NumPy), RAM.

Promotor. Rozszerzenie systemu — zgodne z ideą „nie od zera”.

3.4.10. Graf współwystąpień produktów z koszyków (P10)

Cel i sens. Zbudować graf (węzły = produkty, krawędzie = współwystąpienia w zamówieniach z `seed_eder`) i rekomendować sąsiadów / random walk — klasyczna, jasna linia badawcza bez odwołania do rozmytości czy CBF.

Pytanie badawcze. Czy proste scoringi na grafie (adamic-adar, preferential attachment) ustawiają sensowny ranking w teście czasowym?

Dane, porównania. `seed_research`; baseline popularności.

Co mierzymy. $NDCG@K$, degree distribution grafu, czas budowy.

Promotor. Treść do uzgodnienia z promotorem (graf jako wkład teoretyczno-praktyczny).

3.4.11. Reprodukowalność eksperymentów i automatyzacja (P11)

Cel i sens. Uczynić przewodnim elementem pracy **workflow**: Makefile lub CI, jednolita komenda „od zera do raportu”, wersjonowanie hash `seed_research`, artefakty CSV/JSON — żeby obrona i zespół miały dowód powtarzalności.

Pytanie badawcze. Czy pełny łańcuch (`seed` → przeliczenia → ewaluacja) daje bit-identyczne lub statystycznie zbieżne wyniki?

Dane, porównania. Kilka maszyn lub runów CI; ten sam commit + `seed`.

Co mierzymy. Czas pipeline’u, checksumy wyników, ewentualnie rozbieżności.

Promotor. Możliwy jako wkład inżynierski — do uzgodnienia.

3.4.12. Dryf jakości i monitoring rankingu (P12)

Cel i sens. Zsyntetycznie **zmienić** rozkład person lub intensywność zakupów w kolejnej generacji `seed_research` i zmierzyć, jak bardzo spadają metryki ustalonego rankera — motyw praktyczny pod monitorowanie modeli w prod.

Pytanie badawcze. Które metryki najszybciej wykrywają dryf (np. NDCG vs. coverage)?

Dane, porównania. Dwa manifesty seedera (wersja A vs. B); ten sam kod rankera.

Co mierzymy. Δ metryk, ewentualnie testy na podzbiorach.

Promotor. Opcjonalny temat aplikowany — akceptacja promotora.

4. Przykładowa realizacja wybranych tematów (głęboki opis)

Uwaga. Poniżej rozwinięto **szczegółowy** przykład realizacji odpowiadający m.in. **D1–D3** (Olko) oraz **P2 i P7** (Smoła) z tabel katalogu. Można go zastąpić innym zestawem (np. sam **D9 + P9**, lub **D6 + P11**) przy zachowaniu wspólnej infrastruktury (seed_research, metryki, ExperimentRun).

Po analizie korespondencji od promotora (kombinacja metod, LLM, IT2-FLS, segmentacja — wszystkie zaakceptowane), własnych metod inżynierskich oraz realistyczności w okresie pracy magisterskiej (12–15 mc) rekomendujemy podział, w którym każdy bierze 2 obszary tak, aby **nie przeszkadzać sobie nawzajem**, ale dzielić wspólną infrastrukturę (seeder + ewaluacja + ExperimentRun).

4.1. Praca Dawida Olko — CF + Sentyment + Apriori

Tytuł roboczy

„Hybrydowe łączenie metod rekomendacji z udziałem dużych modeli językowych w systemie e-commerce”

ang.: *Hybrid Recommendation Method Combination with Large Language Models in an E-commerce System*

Pytanie badawcze

Czy kombinacja klasycznych metod rekomendacyjnych (CF + Sentyment + Apriori) ze sobą oraz z modelem LLM daje istotnie lepsze rekomendacje niż każda metoda osobno? Czy LLM rozwiązuje problem zimnego startu lepiej niż CF + Apriori?

Hipotezy

- **H1:** Fuzja rankingowa (RRF / Borda / weighted average) z 3 metod (CF + Sentyment + Apriori) poprawia NDCG@10 o $\geq 10\%$ względem najlepszego pojedynczego rankera.
- **H2:** Dla użytkowników z < 3 zamówieniami w historii LLM (Bielik 11B lub Mistral Small) daje wyższy NDCG@10 niż klasyczne CF i Apriori.
- **H3:** Hybryda LLM + klasyka (LLM jako jeden z rankerów w fuzji) przewyższa zarówno czysty LLM jak i czystą klasykę.
- **H4:** Hallucination rate (rekomendacje produktów spoza katalogu) dla LLM jest niezerowe i zależy od wielkości modelu.

Co konkretnie zrobić w kodzie

Część A — Refactor metod do wspólnego interfejsu:

```
# backend/home/experiments/base_recommender.py
from abc import ABC, abstractmethod

class BaseRecommender(ABC):
    @abstractmethod
    def recommend(self, user_id: int, k: int = 10) -> list[tuple[int, float]]:
        """Zwraca [(product_id, score), ...] długości k, sortowane malejąco."""
```

```

class CollaborativeFilteringRecommender(BaseRecommender):
    def recommend(self, user_id, k=10):
        # opakowuje istniejące process_collaborative_filtering()
        ...

class SentimentRecommender(BaseRecommender): ...
class AprioriRecommender(BaseRecommender): ...

```

Część B — Strategie fuzji rankingów:

```

# backend/home/experiments/fusion.py
class ReciprocalRankFusion:
    def __init__(self, recommenders, k_const=60):
        self.recommenders = recommenders
        self.k_const = k_const

    def recommend(self, user_id, k=10):
        scores = defaultdict(float)
        for r in self.recommenders:
            ranking = r.recommend(user_id, k=50)
            for rank, (pid, _) in enumerate(ranking, start=1):
                scores[pid] += 1.0 / (self.k_const + rank)
        return sorted(scores.items(), key=lambda x: -x[1])[:k]

class BordaCount: ... # sumowanie pozycji
class WeightedAverage: ... # ważona średnia score'ów (po normalizacji)
class Switching: ... # wybór metody zależny od profilu użytkownika

```

Część C — LLM channel:

```

# backend/home/experiments/llm_recommender.py
class LLMRecommender(BaseRecommender):
    def __init__(self, model_name="bielik-11b-v3.0-instruct", mode="zero_shot"):
        # mode in {"zero_shot", "few_shot", "rag"}
        self.client = ollama.Client()
        self.model_name = model_name
        self.mode = mode

    def recommend(self, user_id, k=10):
        # 1. Pobierz historię zakupów użytkownika
        # 2. Pobierz listę kandydatów (np. top-100 z CBF)
        # 3. Build prompt: "Użytkownik kupił X, Y, Z. Mając kandydatów [...],
        #   zarekomenduj 10 produktów uszeregowanych malejąco, w formacie JSON
        #   ."
        # 4. Wywołaj LLM, sparsuj listę produktów
        # 5. Zwaliduj że ID są w katalogu (hallucination check)
        ...

```

Wybór modeli (open-source):

- **Bielik v3 11B** (polski, Apache 2.0) — kluczowy ze względu na polskie nazwy produktów i opisy
- **Mistral Small 3 24B** (Apache 2.0) — referencyjny zachodni

- **Qwen 2.5 14B** (Apache 2.0) — referencyjny azjatycki
- Uruchomienie: **Ollama** (MacBook Pro M3 Pro / Apple Silicon lub PC z GPU; na M3 Pro typowo modele 7B–8B w akceptowalnym czasie)

Część D — Eksperymenty (zapisywane do ExperimentRun):

ID	Konfiguracja	Liczba runów
E1.1–E1.3	CF / Sentyment / Apriori — pojedynczo	3
E1.4–E1.6	Pary: CF+Sentyment, CF+Apriori, Sentyment+Apriori (RRF)	3
E1.7	Trójka CF+Sentyment+Apriori (RRF)	1
E1.8–E1.10	Trójka — Borda, Weighted, Switching	3
E2.1–E2.3	Czysty LLM (Bielik 7B, Bielik 11B, Mistral Small)	3
E2.4–E2.5	LLM zero-shot vs few-shot (best Bielik)	2
E2.6	LLM RAG (Bielik + retrieval z bazy opinii)	1
E3.1–E3.3	Hybryda LLM + klasyka (LLM jako 4. ranker w RRF)	3
Razem		~19 runów × 5 CV ≈ 95 eksperymentów

Tabela 7: Plan eksperymentalny pracy Olko.

Stack technologiczny — Olko

Element	Technologia
LLM inference	Ollama (lokalnie, w tym Apple Silicon); bez vLLM/klastra — nieplanowane w tej pracy
Embeddings (RAG)	sentence-transformers, paraphrase-multilingual-mpnet-base-v2
Vector DB	Qdrant (lekki, lokalny Docker)
Statystyka	scipy.stats — Friedman + post-hoc Nemenyi
Tracking	tabela ExperimentRun + CSV (bez MLflow)
Wizualizacja	matplotlib, seaborn, heatmapa synergii 3×3

Innowacyjność pracy Olko

1. **Polskie modele LLM** — Bielik v3 jest praktycznie nieobecny w literaturze rekomendacyjnej (stan na 01/2026). Praca uzupełnia tę lukę.
2. **Praktyczne porównanie 4 strategii fuzji** — większość artykułów testuje 1–2 strategie.
3. **Hallucination rate** w kontekście rekomendacji produktowych — nowa metryka.
4. **Reprodukowalny seed** — dane testowe dostępne razem z pracą (open science).

Kluczowa literatura — Olko

- Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. UMUAI 12(4).
- Sarwar, B. et al. (2001). Item-based collaborative filtering recommendation algorithms. WWW '01.
- Wu, L. et al. (2024). A Survey on Large Language Models for Recommendation. arXiv:2305.19860.

- Bao, K. et al. (2023). TALLRec: Tuning Framework to Align LLM with Recommendation. RecSys '23.
- Hou, Y. et al. (2024). LLMs are Zero-Shot Rankers for Recommender Systems. ECIR '24.
- Lewis, P. et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP. NeurIPS.
- Ociepa, K. et al. (2026). Advancing Polish Language Modeling — Bielik v3.

4.2. Praca Piotra Smoły — CBF + Fuzzy + Probabilistic

Tytuł roboczy

„Segmentacja użytkowników i wnioskowanie przedziałowe (IT2-FLS) w hybrydowym systemie rekomendacji produktów”

ang.: *User Segmentation and Interval Type-2 Fuzzy Inference in a Hybrid Product Recommendation System*

Pytania badawcze

1. Czy podział użytkowników na segmenty (K-Means / DBSCAN) i osobny dobór wag metod CBF / Fuzzy / Probabilistic dla każdego segmentu poprawia jakość rekomendacji względem konfiguracji globalnej?
2. Czy wnioskowanie rozmyte typu drugiego (IT2-FLS, biblioteka **pyit2fls** wskazana przez promotora) — które uwzględnia niepewność funkcji przynależności — daje lepsze rekomendacje niż obecny system Mamdani T1, szczególnie w warunkach rzadkich danych?

Hipotezy

- **H1:** Strategia świadoma segmentacji przewyższa najlepszą globalną konfigurację o $\geq 3\%$ w NDCG@10.
- **H2:** Optymalna liczba segmentów dla zbioru testowego mieści się w przedziale 4–7 (zgodnie z RFM).
- **H3:** IT2-FLS daje wyższą Precision@10 niż T1 dla użytkowników z < 5 ocenami.
- **H4:** Szerokość przedziału ufności (output IT2-FLS) koreluje istotnie z błędem predykcji.
- **H5:** Koszt obliczeniowy IT2-FLS (defuzy Karnik–Mendel) jest $5\text{--}20\times$ wyższy niż T1, ale opłacalny tylko w cold-start.

Co konkretnie zrobić w kodzie

Część A — Feature engineering użytkowników:

```
# backend/home/experiments/user_features.py
def build_user_feature_vector(user) -> np.ndarray:
    return np.array([
        avg_order_value(user),          # 1. RFM-Monetary
        purchase_frequency(user),        # 2. RFM-Frequency
        category_entropy(user),          # 3. Shannon nad kategoriami
        review_to_order_ratio(user),     # 4. ile pisze opinii
        avg_sentiment_of_opinions(user), # 5. tonacja
        avg_dwell_time_ms(user),         # 6. UserInteraction.duration_ms
        recency_days(user),              # 7. RFM-Recency
        seasonality_score(user),         # 8. jak bardzo sezonowy
        active_hour_avg(user),           # 9.
    ])
```

Część B — Klasteryzacja:

```
# backend/home/experiments/segmentation.py
class UserSegmentation:
    def __init__(self, method="kmeans"):
        self.method = method

    def fit(self, X):
        if self.method == "kmeans":
            best_k, best_model = self._find_best_kmeans(X, k_range=range(2, 11))
            self.model = best_model
        elif self.method == "dbscan":
            self.model = DBSCAN(eps=self._auto_eps(X), min_samples=5).fit(X)
        elif self.method == "gmm":
            best_n, best_model = self._find_best_gmm(X, n_range=range(2, 11))
            self.model = best_model

    def predict(self, X):
        return (self.model.predict(X) if self.method != "dbscan"
                else self.model.fit_predict(X))
```

Część C — Optymalizacja wag per-segment:

```
# backend/home/experiments/segment_optimizer.py
class PerSegmentOptimizer:
    """Dla każdego segmentu znajduje najlepsze wagi (w_cbf, w_fuzzy, w_prob)."""

    def fit(self, segments_dict, train_orders):
        for cluster_id, user_ids in segments_dict.items():
            best_weights = self._grid_search_weights(user_ids, train_orders)
            self.weights[cluster_id] = best_weights

    def recommend(self, user_id, k=10):
        cluster = self.cluster_of(user_id)
        w_cbf, w_fuzzy, w_prob = self.weights[cluster]
        s_cbf = self.cbf.score(user_id)
        s_fuzzy = self.fuzzy.score(user_id)
        s_prob = self.prob.score(user_id)
        final = w_cbf * s_cbf + w_fuzzy * s_fuzzy + w_prob * s_prob
        return top_k(final, k)
```

Część D — IT2-FLS jako alternatywa Mamdani T1:

```
# backend/home/it2_fuzzy_logic_engine.py
import pyit2fls

class IT2RecommendationEngine:
    """Wnioskowanie rozmyte typu drugiego dla scoringu produktów.

    Wymiary wejściowe:
    - cena: tani / średni / drogi (IT2FS z FOU = +/- 20% wzgl. T1)
    - jakość: niska / średnia / wysoka (sentiment-based)
    - popularność: niska / średnia / wysoka (zakupy w okresie)

    Wyjście:
```

```

- score: niski / średni / wysoki (przedział [low, high])
"""

def __init__(self):
    # Footprint of Uncertainty (FOU)
    self.cena_tani = pyit2fls.IT2FS_Gaussian_UncertMean(...)
    self.cena_sredni = pyit2fls.IT2FS_Gaussian_UncertMean(...)
    # ...
    self.system = pyit2fls.IT2Mamdani(...)
    self._add_rules()

def score_product(self, product, user):
    cena_norm = normalize_price(product.price)
    jakosc = sentiment_summary(product)
    popularnosc = buy_count_30d(product)
    # Defuzyfikacja Karnik-Mendel
    result = self.system.evaluate({
        "cena": cena_norm, "jakosc": jakosc, "popularnosc": popularnosc
    })
    return result["score"]

def score_with_uncertainty(self, product, user):
    result = self.system.evaluate(...)
    return result["score_low"], result["score_high"]

```

Część E — Eksperymenty:

ID	Konfiguracja	Liczba runów
E1.1–E1.3	CBF / Fuzzy T1 / Probabilistic — pojedynczo	3
E2.1	Globalna kombinacja CBF+Fuzzy T1+Prob (najlepsze wagi)	1
E2.2–E2.4	K-Means k=4/5/6 — weighted CBF+Fuzzy+Prob per segment	3
E2.5	DBSCAN — outlier detection + fallback to global	1
E2.6	GMM 5 klastrów	1
E3.1	IT2-FLS pojedynczo	1
E3.2	IT2-FLS + CBF + Prob (per segment)	1
E3.3	T1 vs IT2-FLS — sparowane porównanie na 3 segmentach	3
E3.4	Pomiar szerokości CI vs błąd predykcji	1
Razem		~15 runów × 5 CV ≈ 75 eksperymentów

Tabela 8: Plan eksperymentalny pracy Smoły.

Stack technologiczny — Smoła

Element	Technologia
Klasteryzacja	scikit-learn (KMeans, DBSCAN, GaussianMixture)
Redukcja wymiarów	scikit-learn PCA, umap-learn
Walidacja klastrów	Silhouette, Davies-Bouldin, Calinski-Harabasz
IT2-FLS	pyit2fls (<code>pip install pyit2fls</code>) — wskazane przez promotora
Numeryka	NumPy, SciPy
Wizualizacja	matplotlib (membership, FOU plots), seaborn
Statystyka	scipy.stats — sparowany Wilcoxon

Innowacyjność pracy Smoły

1. **Zastosowanie IT2-FLS w rekomendacjach** — w polskim środowisku akademickim praktycznie nieobecne.
2. **Empiryczne porównanie T1 vs T2 fuzzy w cold-start** — odpowiedź na pytanie „czy złożoność IT2 się opłaca”.
3. **Confidence-aware ranking** — produkty z szerokim przedziałem ufności są deprezjonowane lub oznaczane do uwagi użytkownika.
4. **Per-segment weight tuning** — automatyczne mapowanie segment → konfiguracja, rzadko spotykane systemowo.

Kluczowa literatura — Smoła

- Pazzani, M. & Billsus, D. (2007). Content-Based Recommendation Systems. The Adaptive Web.
- Mendel, J. M. (2017). Uncertain Rule-Based Fuzzy Systems, wyd. 2. Springer.
- Karnik, N. N. & Mendel, J. M. (2001). Centroid of a type-2 fuzzy set. Information Sciences.
- Wu, D. (2013). Approaches for reducing the computational cost of IT2-FLS. IEEE TFS.
- Haghray, A. A. & Ghaemi, S. (2023). PyIT2FLS: A New Python Toolkit for IT2-FLS. arXiv:1909.10051.
- MacQueen, J. (1967). Some methods for classification and analysis. Berkeley Symp.
- Ester, M. et al. (1996). DBSCAN. KDD.
- Hennig-Thurau, T. et al. (2012). Automated Group Recommender Systems. J. of Marketing.

5. Wspólny rdzeń projektu

Kawałki, które każdy z autorów musi mieć w swojej pracy — warto zaimplementować raz i podzielić się autorstwem rozdziału (np. „Rozdział: Infrastruktura badawcza” w obu pracach z odpowiednimi referencjami do siebie nawzajem).

5.1. seed_research — szczegółowy plan

Plik: backend/home/management/commands/seed_research.py

```
class Command(BaseCommand):
    help = "Generuje powtarzalny zbiór badawczy z personami."

    def add_arguments(self, parser):
        parser.add_argument("--clients", type=int, default=200)
        parser.add_argument("--admins", type=int, default=5)
        parser.add_argument("--products", type=int, default=500)
        parser.add_argument("--days-back", type=int, default=730)
        parser.add_argument("--orders-per-client-mean", type=int, default=12)
        parser.add_argument("--interactions-per-client-mean", type=int, default=80)
        parser.add_argument("--random-seed", type=int, default=42)
        parser.add_argument("--reset", action="store_true",
                              help="Wyczyść bazę przed seedowaniem")

    def handle(self, *args, **opts):
        random.seed(opts["random_seed"])
        np.random.seed(opts["random_seed"])
        if opts["reset"]:
            self._drop_research_data()
            # 1. Produkty (zachowujemy z istniejącego seeda)
            self._ensure_products(opts["products"])
            # 2. Persony
            personas = self._distribute_personas(opts["clients"])
            # 3. Użytkownicy z personami (zapis do UserPersona)
            users = self._create_users(personas, opts["admins"])
            # 4. Zamówienia (temporalnie spójne, persona-driven)
            self._create_orders(users, days_back=opts["days_back"])
            # 5. Interakcje (view/click przed zakupem)
            self._create_interactions(users)
            # 6. Opinie (po zakupie, sentyment-persona-correlated)
            self._create_opinions(users)
            # 7. Sentiment summary, similarity, association rules
            self._recompute_all_methods()
            # 8. Metadane seedowania
            self._log_seed_metadata(opts)
```

5.2. home/experiments/ — wspólny moduł

```
backend/home/experiments/
|- __init__.py
```

```
| - base_recommender.py      # BaseRecommender + adaptery do istniejących metod
| - metrics.py               # precision@k, recall@k, ndcg@k, map, coverage
| - splits.py                # temporal_split, leave_last_one_out, k_fold_user
| - runner.py                # ExperimentRunner      uruchamia, mierzy, loguje
| - reporter.py              # generator raportów CSV i wykresów
\ - tests/
    | - test_metrics.py
    \ - test_splits.py
```

Wspólne API:

```
from home.experiments import temporal_split, ExperimentRunner

train, test = temporal_split(test_ratio=0.2)
runner = ExperimentRunner(method=my_recommender, k=10)
results = runner.evaluate(test_users=test["users"],
                           ground_truth=test["orders"])
runner.save_to_db(experiment_id="E1.1_CF_solo", notes="Baseline CF")
```

5.3. Tracking dwell time (frontend + backend) — opcjonalny

Implementujemy tylko jeśli Smoła używa `dwell_time_mean_ms` w `build_user_feature_vector()`.

Backend: migracja `UserInteraction.duration_ms`, rozszerzenie `POST /api/interaction/` o pole `duration_ms`.

Frontend (`frontend/src/components/ProductSection/ProductPage.jsx`):

```
useEffect(() => {
  const enterTime = Date.now();
  return () => {
    const durationMs = Date.now() - enterTime;
    axios.post('/api/interaction/', {
      product: productId,
      interaction_type: 'view',
      duration_ms: durationMs
    });
  };
}, [productId]);
```

Drobne zmiany, kilka godzin pracy. Może być wyłączone feature flagiem `REACT_APP_TRACK_DWELL=true` w produkcji.

5.4. Panel diagnostyczny eksperymentów (opcjonalnie)

W panelu admina dodać kartę „Eksperymenty” w `frontend/src/components/AdminPanel/`:

- Lista zdefiniowanych eksperymentów (z `ExperimentRun.experiment_id`)
- Wykres porównawczy `NDCG@10` dla wybranego zbioru
- Eksport do CSV
- Przycisk „Uruchom” wyzwalający Django management command w tle

To opcjonalne — eksperymenty można uruchamiać z terminala. Panel UI dodaje wartości jeśli promotor chce zobaczyć demo na obronie.

6. Tabelaryczne porównanie tematów

Aspekt	Olko (Konfig. A)	Smoła (Konfig. B)
Tytuł skrócony	Hybrydy + LLM	Segmentacja + IT2-FLS
Bazuje na	CF / Sentyment / Apriori	CBF / Fuzzy / Probabilistic
Główne pytanie	Czy fuzja + LLM > klasyka?	Czy segmenty + IT2-FLS > globalna T1?
Liczba hipotez	4	5
Liczba eksperymentów	~95	~75
Najtrudniejszy element	Dobór rozmiaru modelu LLM i czasu inferencji wyłącznie na laptopie (M3 Pro)	Implementacja IT2-FLS dla 3 wymiarów
Wymagania sprzętowe	Własny MacBook Pro M3 Pro (brak GPU/klastra na uczelni); ewentualnie zwężenie eksperymentu	Standardowy laptop, IT2-FLS jest CPU-bound
Ryzyka	Hallucinations LLM, długi czas inferencji	Złożoność defuzy KM, czasochłonny grid search wag
Wsparcie literaturowe	bardzo dobre (Da'u 2025, Bao 2023, Wu 2024)	dobrze (Mendel 2017, Karnik–Mendel 2001)
Stanowisko promotora	LLM „interesujące, na topie” + kombinacja „dobra ścieżka”	IT2-FLS „moja propozycja” + segmentacja „warto”

Tabela 9: Porównanie tematów Olko vs Smoła.

7. Harmonogram realizacji (12 miesięcy)

Mies.	Kamień milowy	Wspólne	Olko	Smoła
1	Infrastruktura	seed_research v1, ExperimentRun, metrics module	—	—
2	Refactor metod	adaptery BaseRecommender	CF/Sent/Apr → Recommender	CBF/Fuzzy/Prob → Recommender
3	Baseline	—	E1.* (single)	E1.* (single)
4–5	Główne metody	—	Fuzja + LLM channel	Segmentacja + IT2-FLS
6–7	Eksperymenty	—	E2.* , E3.*	E2.* , E3.*
8	Analiza statystyczna	wspólne testy Friedman	wykresy synergii	wykresy CI vs error
9–10	Pisanie pracy	—	rozdziały 1–7	rozdziały 1–7
11	Dopracowanie	—	review koleżeńskie	review koleżeńskie
12	Obrona	—	—	—

Tabela 10: Harmonogram pracy magisterskiej dla obu autorów.

8. Ryzyka i plany awaryjne

Ryzyko	Prawdop.	Plan B
LLM nie chce się uruchomić lokalnie (Olko)	niskie	Mniejszy model (7B), krótszy kontekst, mniejsza próba testowa; ewentualnie dłuższy czas batcha w nocy na M3 Pro
pyit2fls okazuje się buggy (Smoła)	niskie	Implementacja własna IT2-FLS na NumPy + Karnik–Mendel z literatury
Dane są zbyt mało zróżnicowane	niskie	Zwiększenie skali seedera do 500 użytkowników, dodanie person
Eksperymenty trwają zbyt długo	średnie	Próbkowanie testowe 50% użytkowników, zwężenie CV
Promotor zmienia zdanie co do podziału	niskie	Konfiguracja modułowa — każdy temat samodzielny

Tabela 11: Ryzyka i plany awaryjne.

9. Założenia środowiskowe — część LLM (Olko), ustalone przez autora

- **Brak zaplecza na uczelni** (GPU, klaster) — całość eksperymentów z udziałem LLM jest realizowana na **własnym laptopie MacBook Pro M3 Pro**.
- **Modele wyłącznie lokalne i otwarte** (priorytet: **Bielik** i ewentualnie inne open-weight przez Ollama) — **bez** komercyjnych API (GPT-4 itd.) jako podstawy pracy. Uzasadnienie wobec promotora: **reprodukowalność** (ustalone środowisko), **brak kosztów tokenów**, **polski model** jako świadomy wkład merytoryczny, pełna kontrola nad wersją wag i seedem eksperymentu.
- API płatne można wymienić w pracy wyłącznie jako teoretyczną alternatywę porównawczą, nie jako plan realizacji.

10. Pytania do uzgodnienia z promotorem

1. Czy akceptowalna jest **pula tematów** z sekcji 3, a jako głęboki szablon realizacji — przykładowy wariant z sekcji 4 (Olko: LLM + kombinacja; Smoła: segmentacja + IT2-FLS), ewentualnie zamiana na inną parę pozycji z katalogu?
2. Czy promotor przewiduje publikację wyników (artykuł na konferencję, np. PP-RAI, FedCSIS)? Wpłynie to na język pisania (pl/en) i staranność eksperymentu.
3. Czy rozszerzyć UI o panel diagnostyczny eksperymentów (przyciski „Uruchom E1.1”, live preview wyników) czy zostać przy management commands z terminala?
4. Jaka jest minimalna i maksymalna oczekiwana objętość pracy magisterskiej (liczba stron) w Pani/Pana wymaganiach?

Dokument przygotowany jako finalna rekomendacja po analizie istniejącej bazy kodu, korespondencji z promotorem oraz dwóch wcześniejszych dokumentów propozycji. Wszystkie wskazane interwencje w kodzie zostały zweryfikowane na obecnej strukturze projektu.